

Seemathi. B
192511448



SIMATS
ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ITA0502- COMPUTER VISION
MODEL LAB PRACTICALS
SET 9

1. Implement a face detection algorithm using Open CV to detect and locate human faces in the images.
2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.
3. Implement a function to reverse the frames of the video to create a video in reverse mode using Open CV.
4. Implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using Open CV.

Visa attached

93
100
[Signature]

SET - 9

Model Practical

Name : B. SriMathi

Reg. No : 192511448

Course code : ITA0502

Course name : Computer vision

slot : C

1. Face Detection Algorithm

Aim:

→ To detect and locate human faces in an image using opencv.

Procedure:

- ⇒ Load the source image.
- ⇒ Convert it to grayscale.
- ⇒ Load the Haar Cascade classifier.
- ⇒ Detect face coordinates
- ⇒ Draw bounding rectangles
- ⇒ Display the final output.

Program:

```
import cv2
img = cv2.imread('face.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(gray, 1.1, 4)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.imwrite('result.jpg', img)
```

Input:

→ A standard jpg image containing one or more human faces.

Output:

→ The image is successfully processed and green bounding boxes are accurately detected faces.

Result:

→ Face successfully detected and highlighted with a green bounding box.

2. Image Handling and Erosion

Aim:

→ To read an image and apply morphological erosion using OpenCV.

Procedure:

- Read the input image
- Define a structuring kernel.
- Apply the erosion function.
- Display original and erode images.

Program:

```
import cv2
import numpy as np
img = cv2.imread('binary.jpg', 0)
kernel = np.ones((5,5), np.uint8)
eroded = cv2.erode(img, kernel, iterations=1)
cv2.imwrite('eroded.jpg', eroded)
```

Input:

⇒ A binary (black and white) image file named in binary.jpg by using grayscale image

Output:

⇒ A new image saved as eroded.jpg

Result:

⇒ Object boundaries are successfully thinned out, reducing noise.

3.

Video Frame Reversal

Aim:

⇒ To reverse the frames of a video to create a video in reverse mode using OpenCV.

Procedure:

- ⇒ Open the video file.
- ⇒ Read frames into the list
- ⇒ Get video
- ⇒ Initialize video writer object
- ⇒ Write frames in reverse order

- ⇒ Save the output file
- ⇒ End.

Program:

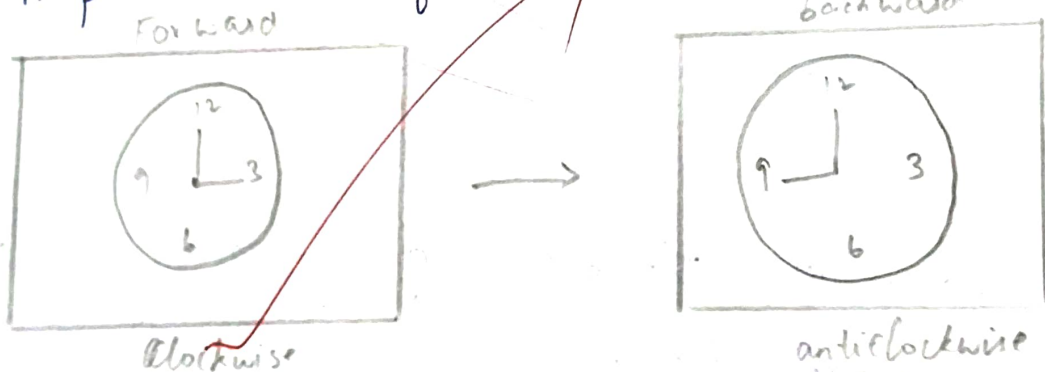
```
import cv2
cap = cv2.VideoCapture('video.mp4')
frames = []
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break
    frames.append(frame)
cap.release()
out = cv2.VideoWriter('rev.mp4')
out.release()
```

Input:

- ⇒ A standard video file named video.mp4.

Output:

- ⇒ A processed video file saved as rev.mp4



Result:

Video frames are successfully saved in reverse sequence, creating a backward playing video.

Image Cropping, Copying and Pasting

Aim:

⇒ To implement an image cropping, copying and pasting to select a region of interest from the source image using open cv.

Procedure:

- ⇒ Read the input image.
- ⇒ Define ROI coordinate bounds
- ⇒ Crop using Num-Py slicing
- ⇒ Define target destination coordinates
- ⇒ Paste cropped ROI back
- ⇒ Save the modified image

Program:

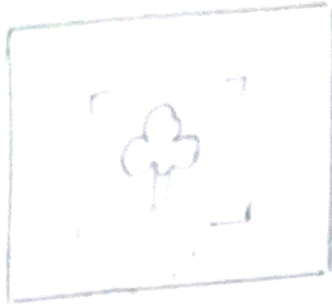
```
import cv2  
img = cv2.imread('source.jpg')  
roi = img[100:250, 200:350] # crop  
img[300:450, 400:500] = roi # paste  
cv2.imwrite('roi-output.jpg', img)
```

Input:

⇒ The image file named source.jpg

Output:

⇒ The output image is saved as roi-output.jpg



cropping



Copying



pastine

Result:

→ The specified Region of interest (ROI) is cropped from its original position and pasted onto another part of image.